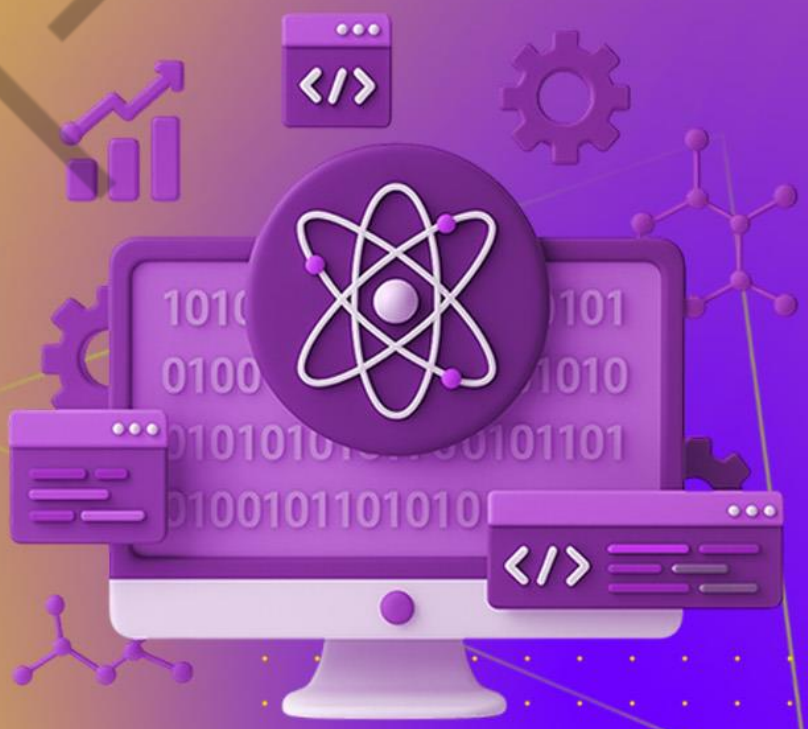
SUPERANDO LIMITES: ERA DA MOBILIDADE E VISÃO INTELIGENTE

QUBIT POWER: CIRCUITOS, ENTRELAÇAMENTO E A MATÉRIA QUÂNTICA



4

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

LISTA DE FIGURAS

Figura 1 - Porta controlada customizada em Qiskit 15
Figura 2 - Porta controlada customizada com circuito 16
Figura 3 - Portas CNOT 18
Figura 4 - Porta Toffoli decomposta 20
Figura 5 - Portas CCX 21
Figura 6 - Porta SWAP decomposta 22
Figura 7 - Porta SWAP 22
Figura 8 - Porta CSWAP decomposta 24
Figura 9 - Porta CSWAP decomposta com Toffoli 25
Figura 10 - Porta CSWAP 25
Figura 11 - Resultados da simulação 36
Figura 12 - Circuito Phi plus 40
Figura 13 - Circuitos para as Bases de Bell 41
Figura 14 - Circuito GHZ de 5 qubits 42
Figura 15 - Circuito W de 4 qubits 45
Figura 16 - Medições do Circuito W de 4 qubits 45
Figura 17 - Medições do Circuito GHZ com erros 47
Figura 18 - Código de repetição para detecção de bit-flip 49
Figura 19 - Código de repetição para detecção de phase-flip 50
Figura 20 - Código para detecção de phase-flip adaptado 50
Figura 21 - Código de Shor 52

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

LISTA DE QUADROS

Quadro 1 - Síndromes para bit-flip 49
Quadro 2 - Síndromes para phase-flip 51

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Criando portas controladas com Qiskit 15
Código-fonte 2 - Criando portas controladas com circuitos 16
Código-fonte 3 - Circuito da porta Toffoli decomposta 20
Código-fonte 4 - Circuito da porta SWAP decomposta 21
Código-fonte 5 - Circuito da porta CSWAP decomposta 24
Código-fonte 6 - Simulação com Qiskit Aer 35
Código-fonte 7 - Identificando inversão de fase com interferência 38
Código-fonte 8 - Circuito Phi plus 40
Código-fonte 9 - Circuito GHZ de 5 qubits 42
Código-fonte 10 - Circuito W de 4 qubits 44
Código-fonte 11 - Circuito GHZ com erros 47

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

SUMÁRIO

1 QUBIT POWER: CIRCUITOS, ENTRELAÇAMENTO E A MATÉRIA QUÂNTICA.6
1.1 Introdução 6
2 SISTEMAS QUÂNTICOS MULTI-QUBITS.....7
2.1 Medição de estados compostos 9
2.2 Medição parcial de estados compostos.....10
2.3 Resultado da medição parcial 11
3 PORTAS LÓGICAS MULTI-QUBITS 13
3.1 Operadores locais 13
3.2 Operadores Globais 14
3.3 Operadores Unitários Controlados 14
3.4 Portas Multi-Qubits importantes 16
3.4.1 Porta CNOT.....16
3.4.2 Porta Toffoli 18
3.4.3 Porta SWAP 21
3.4.4 Porta Fredkin.....22
4 ESTADOS ENTRELAÇADOS 26
4.1 Paradoxo EPR.....26
4.2 Estados entrelaçados importantes 29
4.2.1 Estados de Bell 30
4.2.2 Estado GHZ.....31
4.2.3 Estado W.....32
5 CIRCUITOS QUÂNTICOS MULTI-QUBITS 34
5.1 Simulação com Qiskit Aer 34
5.2 Interferência e Circuito HZH.....36
5.3 Circuitos de estados emaranhados.....38
5.3.1 Estados de Bell 39
5.3.2 Estado GHZ.....41
5.3.3 Estado W.....42
6 INTRODUÇÃO À CORREÇÃO DE ERROS QUÂNTICOS 46
6.1 Simulação de erros com Qiskit Aer 46
6.2 Princípios da correção de erros.....48
6.2.1 Código de Repetição e Bit-Flip.....48
6.2.2 Adaptando para corrigir Phase-Flip.....50
6.3 Código de Shor 51
CONCLUSÃO.....54
REFERÊNCIAS.....55

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

1 QUBIT POWER: CIRCUITOS, ENTRELAÇAMENTO E A MATÉRIA QUÂNTICA

1.1 Introdução

Nos capítulos anteriores, definimos o qubit como a unidade fundamental da Computação Quântica, demonstrando sua representação na Esfera de Bloch, operadores unitários e seus usos em circuitos simples com um único qubit ou com qubits independentes.

Neste capítulo, expandiremos os conceitos ao apresentar **sistemas com múltiplos qubits** relacionados entre si. Ao lidar com qubits conectados, novas propriedades surgem, em especial o **entrelaçamento quântico**, outro fenômeno intrinsecamente quântico (como a superposição), que serve como base para as vantagens de computadores quânticos sobre os clássicos na execução de algoritmos.

Além disso, introduziremos a **correção de erros quânticos** para reduzir efeitos de **ruído** e de **coerência**, fenômenos que dificultam a criação de algoritmos e a utilização da Computação Quântica no mundo real, gerando instabilidade nos estados e problemas nos circuitos.

Mitigar esses erros é crucial para a utilização prática da Computação Quântica.

Esses tópicos não são apenas a continuação natural do que vem sendo estudado nos capítulos anteriores, mas também servem como ponto inicial para o desenvolvimento de algoritmos mais complexos, como algoritmos de **busca**, de **ordenação**, de **criptografia** e aplicações mais avançadas, como o **Quantum Machine Learning** (QML).

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

2 SISTEMAS QUÂNTICOS MULTI-QUBITS

Assim como não podemos fazer muitas coisas ao trabalhar com informação clássica utilizando apenas um bit, os sistemas quânticos com um único qubit também têm utilidade limitada.

É possível **unir múltiplos sistemas** de um qubit, obtendo um único sistema composto. Matematicamente, representamos o sistema composto como o **produto tensorial** dos sistemas isolados.

Seja dois sistemas quânticos, $|\psi\rangle$ e $|\phi\rangle$, respectivamente possuindo um conjunto de estados $\Sigma = \{0,1\}$ e $\Gamma = \{0,1\}$. O sistema composto por $|\psi\rangle$ e $|\phi\rangle$ será:

$$|\psi\rangle \otimes |\phi\rangle$$

Para simplificar, ao representar um estado composto, podemos omitir o sinal de produto tensorial ou, ainda, unir ambos estados em um único *ket*:

$$|\psi\rangle \otimes |\phi\rangle = |\psi\rangle|\phi\rangle = |\psi\phi\rangle$$

Essa última forma destaca que o estado composto é um vetor de estados contendo todas as permutações dos sistemas isolados. Podemos confirmar ao realizar o produto:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle;\ |\phi\rangle = \gamma|0\rangle + \delta|1\rangle$$

$$\begin{aligned} |\psi\rangle \otimes |\phi\rangle &= (\alpha\,|0\rangle + \beta\,|1\rangle) \otimes (\gamma\,|0\rangle + \delta\,|1\rangle) \\ |\psi\rangle \otimes |\phi\rangle &= (\alpha\gamma\,|0\rangle \otimes |0\rangle) + (\alpha\delta\,|0\rangle \otimes |1\rangle) + (\beta\gamma\,|1\rangle \otimes |0\rangle) + (\beta\delta\,|1\rangle \otimes |1\rangle) \end{aligned}$$

Calculando os produtos entre as bases computacionais, temos que:

$$|0\rangle \otimes |0\rangle = \begin{pmatrix} 1\begin{pmatrix}1\\0\end{pmatrix} \\ 0\begin{pmatrix}1\\0\end{pmatrix} \end{pmatrix} = \begin{pmatrix}1\\0\\0\\0\end{pmatrix} = |00\rangle$$

$$|0\rangle \otimes |1\rangle = \begin{pmatrix} 1\begin{pmatrix}0\\1\end{pmatrix} \\ 0\begin{pmatrix}0\\1\end{pmatrix} \end{pmatrix} = \begin{pmatrix}0\\1\\0\\0\end{pmatrix} = |01\rangle$$

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

$$|1\rangle \otimes |0\rangle = \begin{pmatrix} 0\begin{pmatrix}1\\0\end{pmatrix} \\ 1\begin{pmatrix}1\\0\end{pmatrix} \end{pmatrix} = \begin{pmatrix}0\\0\\1\\0\end{pmatrix} = |10\rangle$$

$$|1\rangle \otimes |1\rangle = \begin{pmatrix} 0\begin{pmatrix}0\\1\end{pmatrix} \\ 1\begin{pmatrix}0\\1\end{pmatrix} \end{pmatrix} = \begin{pmatrix}0\\0\\0\\1\end{pmatrix} = |11\rangle$$

Portanto, o estado composto será equivalente a:

$$|\psi\rangle \otimes |\phi\rangle = \alpha\gamma\,|00\rangle + \alpha\delta\,|01\rangle + \beta\gamma\,|10\rangle + \beta\delta\,|11\rangle$$

$$|\psi\rangle \otimes |\phi\rangle = \begin{pmatrix}\alpha\gamma\\ \alpha\delta\\ \beta\gamma\\ \beta\delta\end{pmatrix}$$

A forma de matriz do vetor também evidencia os estados possíveis para a medição, representados formalmente por $\Sigma \times \Gamma$:

$$\Sigma \times \Gamma = \{(a, b) : \ a \in \Sigma, b \in \Gamma\}$$

No caso das bases computacionais:

$$\Sigma \times \Gamma = \{00, 01, 10, 11\}$$

Podemos formar sistemas de qualquer tamanho utilizando o mesmo conceito. Como a norma é multiplicativa em relação ao produto tensorial (a norma do produto é o produto das normas), ela se mantém, ou seja, o **estado composto** continua **normalizado** e representa um **estado quântico válido**.

Vale destacar que, para estados compostos deste tipo, é possível fazer o caminho inverso: **separar** um sistema que é sabidamente composto nos vetores de estado contendo qubits **individuais**, o que pode auxiliar algumas análises e operações, como a medição.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

2.1 Medição de estados compostos

Calcular a probabilidade de se medir um estado é similar ao método que utilizamos para sistemas de um qubit:

- A amplitude de probabilidade de se medir cada estado equivale ao seu coeficiente, ou seja, a probabilidade de se medir tal estado é o módulo do coeficiente ao quadrado.
- Para se medir um estado em específico, podemos calcular o módulo ao quadrado do **produto interno** entre o estado em questão e o vetor do sistema.

Por exemplo, considere o estado composto $|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$. A partir das amplitudes de probabilidade, vemos que a probabilidade de se medir:

- $|00\rangle$ é $|\alpha|^2$.
- $|01\rangle$ é $|\beta|^2$.
- $|10\rangle$ é $|\gamma|^2$.
- $|11\rangle$ é $|\delta|^2$.

Para calcular a probabilidade $P(x)$ de se medir o estado genérico $|x\rangle$ em $|\psi\rangle$, calculamos o **produto interno**, tal que $P(x) = |\langle x|\psi\rangle|^2$. Por exemplo, a probabilidade de se medir $|00\rangle$ no estado $|\psi\rangle$ descrito acima é:

$$\begin{aligned} P(00) &= |\langle 00|\psi\rangle|^2 = |\langle 00|\alpha\,|00\rangle + \beta\,|01\rangle + \gamma\,|10\rangle + \delta\,|11\rangle\rangle|^2 \\ P(00) &= |\alpha\,\langle 00|00\rangle + \beta\,\langle 00|01\rangle + \gamma\,\langle 00|10\rangle + \delta\,\langle 00|11\rangle|^2 \\ P(00) &= |\alpha|^2 \end{aligned}$$

Outra maneira de se calcular as probabilidades, útil para medir estados arbitrários, é utilizar a **projeção** de estados:

- Definimos um **operador de projeção** Π_x como sendo o produto externo do estado que queremos medir (x) por ele próprio. Esse operador projeta o estado ao qual é aplicado na direção do estado que queremos medir.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

- Utilizando a **regra de Born**, podemos calcular a probabilidade de se medir o estado x por meio da projeção.

Considere que o operador de projeção é $\Pi_x = |x\rangle\langle x|$. Para medir $|x\rangle$ em $|\psi\rangle$, aplicamos a regra de Born, ilustrada a seguir:

$$\langle\psi|\Pi_x|\psi\rangle = \langle\psi|x\rangle\langle x|\psi\rangle = |\langle x|\psi\rangle|^2$$

Dessa forma, é fácil ver que utilizar a projeção é equivalente a medir o estado através do módulo ao quadrado da amplitude de probabilidade.

Para ilustrar, considere o exemplo anterior, em que queremos calcular a probabilidade de se medir o estado $|00\rangle$ em $|\psi\rangle$:

$$\Pi_{00} = |00\rangle\langle 00|$$

$$P(00) = \langle\psi|\Pi_{00}|\psi\rangle = \langle\psi|00\rangle\langle 00|\psi\rangle = |\langle 00|\psi\rangle|^2 = |\alpha|^2$$

Essas relações servem para estados compostos de *n*-qubits, para **qualquer *n*** arbitrário (após as alterações necessárias).

2.2 Medição parcial de estados compostos

Para estados compostos, também podemos realizar **medições parciais**, ou seja, medir apenas um qubit no sistema.

Tal operação é equivalente a calcular a **projeção em um dos qubits** e não fazer nada nos demais, representada pelo produto tensorial entre a matriz **Identidade** (nos qubits que não serão afetados) e o **operador de projeção** (no qubit que queremos medir).

Suponha o sistema composto com dois qubits $|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$. Para calcular a probabilidade de se medir o estado $|0\rangle$ no primeiro qubit (da esquerda).

Primeiro, definimos nosso operador de projeção Π_0:

$$\Pi_0 = |0\rangle\langle 0|$$

Em seguida, o operador de projeção que será aplicado ao sistema. Como queremos aplicar Π_0 no primeiro qubit e não manipular o segundo, teremos:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

$$\Pi_0 \otimes \mathbb{I}$$

Ao aplicar o operador ao sistema, teremos as amplitudes de probabilidade do estado $|0\rangle$ no qubit desejado:

$$(\Pi_0 \otimes \mathbb{I})|\psi\rangle = |0\rangle\langle 0| \otimes \mathbb{I}\,(\alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle)$$

$$(\Pi_0 \otimes \mathbb{I})|\psi\rangle = \alpha|0\rangle\langle 0|0\rangle|0\rangle + \beta|0\rangle\langle 0|0\rangle|1\rangle + \gamma|0\rangle\langle 0|1\rangle|0\rangle + \delta|0\rangle\langle 0|1\rangle|1\rangle$$

$$(\Pi_0 \otimes \mathbb{I})|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + 0 + 0 = \alpha|00\rangle + \beta|01\rangle$$

Com a amplitude, calculamos a probabilidade com seu módulo ao quadrado:

$$P(0 \otimes \mathbb{I}) = |\alpha|00\rangle + \beta|01\rangle|^2 = |\alpha|^2 + |\beta|^2$$

Assim como na medição do sistema completo, esse passo a passo é válido para qualquer sistema composto por múltiplos qubits. Caso haja mais de dois qubits no sistema podemos, ainda, medir múltiplos ao mesmo tempo ao definir o operador adequado.

2.3 Resultado da medição parcial

Além disso, vale destacar o que acontece com o estado do sistema após a medição parcial. Sabemos que a medição colapsa a função de onda, resultando em um estado clássico para o sistema.

No entanto, o tipo de sistema com o qual estamos trabalhando até o momento é composto por **qubits independentes** entre si, de forma que a medição de um qubit **não colapsa** o sistema como um todo.

O vetor estado do sistema após a medição parcial levará em consideração os resultados da medição, mas será **re-normalizado** ao dividi-lo por sua norma.

Considerando o exemplo da seção anterior, no qual medimos o primeiro qubit como zero, o novo estado do sistema, $|\psi'\rangle$, será dado por:

$$|\psi'\rangle = \frac{\alpha|00\rangle + \beta|01\rangle}{\sqrt{|\alpha|^2 + |\beta|^2}}$$

É fácil ver que a probabilidade de se medir o estado $|0\rangle$ no primeiro qubit é de 100%, indicando o colapso desse estado. No entanto, o segundo qubit permanece indefinido.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

De forma geral, o vetor de estado após a medição será sempre o vetor original re-normalizado, ou seja, o vetor de amplitudes de probabilidade calculado inicialmente dividido por sua norma.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

3 PORTAS LÓGICAS MULTI-QUBITS

Assim como é possível estabelecer um paralelo entre a medição de estados compostos e a de estados de qubit único, as portas lógicas que operam sobre sistemas compostos funcionam de maneira similar.

Os operadores também são **matrizes unitárias**, aplicadas ao sistema por meio de uma multiplicação de matrizes. A maior diferença está na forma que definimos as matrizes.

3.1 Operadores locais

Anteriormente, definimos nosso operador de medição parcial como sendo o produto tensorial do operador de projeção pela matriz identidade, representando uma operação na qual medimos um qubit e não fazemos nada no outro.

Um dos tipos de operadores lógicos são definidos da mesma forma: um **produto tensorial** das matrizes que queremos aplicar a cada qubit — chamados **operadores locais**.

Considere um sistema com dois qubits, no qual queremos aplicar a porta X no primeiro e Z no segundo. Para isso, aplicamos ao vetor de estado o seguinte operador:

$$X \otimes Z = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}$$

Esta operação aplicará as portas em cada qubit **isoladamente**, sendo uma das principais propriedades dos operadores locais. Por consequência, ao analisar a matriz resultante, é possível realizar a operação inversa e **separá-la** como o produto tensorial das **portas lógicas originais**, similar aos estados compostos que podem ser separados em um produto tensorial de estados.

Em Qiskit, podemos aplicar operadores locais a um qubit da mesma forma que fizemos no capítulo anterior ao trabalhar com sistemas isolados: basta aplicar diretamente as portas lógicas originais a cada qubit.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

3.2 Operadores Globais

Ao contrário dos operadores locais, podemos definir operações que modificam o sistema composto **como um todo**, conhecidos como **operadores globais**.

Estes **não podem** ser definidos a partir de um produto tensorial, o que exige que a **matriz seja escrita explicitamente**. Para isso, podemos utilizar técnicas como análise da Tabela Verdade que queremos aplicar com a operação ou similares.

No entanto, operadores globais podem ser separados como uma **expansão de Pauli**, ou seja, uma **combinação linear** entre **produtos tensoriais** de **matrizes de Pauli** ($\mathbb{I}, X, Y$ e Z).

Alguns operadores globais importantes são: a porta **SWAP** e as **portas controladas** — como **CNOT**, **Toffoli** e **Fredkin**.

Da mesma forma que os operadores locais, é possível definir um operador global arbitrário em Qiskit a partir de sua matriz, assim como fizemos no capítulo anterior para portas lógicas de um qubit.

3.3 Operadores Unitários Controlados

Operadores unitários controlados são um tipo de porta lógica global que aplicam uma operação unitária em um qubit (chamado **alvo**) com base no estado de um ou mais qubits (chamados **qubits de controle**).

Considerando M uma matriz unitária correspondente a um operador lógico, podemos definir a operação controlada CM, que agirá sobre o qubit Q como:

$$CM = \left(|0\rangle\langle 0| \otimes \mathbb{I}_Q\right) + \left(|1\rangle\langle 1| \otimes M\right)$$

Em uma operação controlada, a porta lógica será aplicada ao qubit alvo caso o bit de controle esteja ativo (ou seja, no estado $|1\rangle$). Não é necessário realizar medições para que portas controladas sejam ativadas.

Nota: no caso de o qubit de controle ter um estado definido, ocorrerá uma interação diretamente entre ele e o alvo, aplicando ou não a operação. Estudaremos o caso de o controlador estar em superposição na próxima seção.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Em Qiskit, há um método específico de se criar operadores controlados. Podemos utilizar os *Gate*s e invocar o método *control*, passando como parâmetro o número de bits de controle.

O próximo código-fonte ilustra a criação de uma porta controlada customizada, que aplica uma porta Hadamard no qubit alvo com base em dois controles.

```
from qiskit import QuantumCircuit
from qiskit.circuit.library.standard_gates import HGate

qc = QuantumCircuit(3)
ch = HGate().control(2)
qc.append(ch, [0, 2, 1])

display(qc.draw('mpl')) # Figura 1
```

Código-fonte 1 - Criando portas controladas com Qiskit
Fonte: Elaborado pelo autor (2025)

A figura ilustra essa porta sendo aplicada em um circuito de 3 qubits:

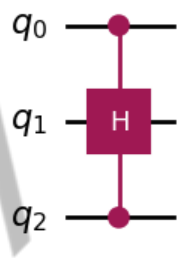


Figura 1 - Porta controlada customizada em Qiskit
Fonte: Elaborado pelo autor (2025)

A representação gráfica de qualquer porta controlada é similar: cada círculo pequeno preenchido identifica um qubit de controle, enquanto a porta lógica é aplicada ao qubit alvo.

Em Qiskit, ao adicionar uma porta controlada ao circuito, podemos passar como argumento o índice do qubit controlador (ou uma lista, em caso de haver vários) e o índice do alvo (ou uma lista). No caso de uma porta customizada, como a do código-fonte anterior, passamos todos os argumentos em uma lista, no qual os *n* primeiros são os índices dos controles e o restante são os alvos.

Um código similar pode ser aplicado para criar qualquer porta controlada arbitrária. Outra forma é criar um operador a partir de um circuito, transformando-o em porta controlada com o mesmo método *control*, como mostra o próximo código-fonte.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

```
from qiskit import QuantumCircuit

qc_op = QuantumCircuit(2)
qc_op.x(0)
qc_op.h(1)
custom_op = qc_op.to_gate().control(2)

qc = QuantumCircuit(4)
qc.append(custom_op, [0, 3, 2, 1])

display(qc.draw('mpl')) # Figura 2
```

Código-fonte 2 - Criando portas controladas com circuitos
Fonte: Elaborado pelo autor (2025)

O resultado do código está ilustrado na figura:

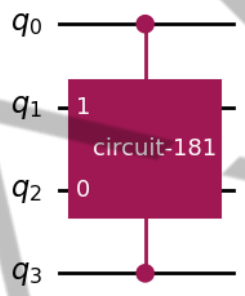


Figura 2 - Porta controlada customizada com circuito
Fonte: Elaborado pelo autor (2025)

O código-fonte "Criando portas controladas com circuitos" ilustra não apenas a criação de uma porta controlada a partir de um circuito, mas também a criação de um operador local, que aplica as operações X e H de forma independente nos qubits alvo.

3.4 Portas Multi-Qubits importantes

Como mencionado anteriormente, algumas portas globais se destacam por suas utilidades ou frequência de uso, especialmente as portas **CNOT**, **Toffoli**, **SWAP** e **Fredkin**.

3.4.1 Porta CNOT

Também conhecida como porta **Feynman** por alguns autores, a porta CNOT tem seu nome originado da abreviação de **Controlled-NOT**. Trata-se de uma porta

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

controlada que aplica uma **negação** (inversão) no qubit alvo, ou seja, a porta X de Pauli.

Mesmo entre as portas em destaque, a CNOT merece uma atenção maior por causa de sua ampla utilidade, incluindo:

- Aplicar operações reversíveis (a partir do resultado da operação, podemos identificar os estados de entrada) sem destruir estados em superposição.
- Criar estados emaranhados.
- Ser a base para implementar operações condicionais em circuitos.
- Aplicações em circuitos de correção de erros.
- Entre várias outras.

Muitas dessas aplicações serão exploradas em mais detalhes ao longo deste e dos demais capítulos.

Vale mencionar também o paralelo com a porta clássica **NAND**. Assim como a NAND é considerada a porta universal da computação clássica — isto é, todas as demais portas podem ser implementadas utilizando apenas uma associação de NANDs —, a **CNOT** é a parte principal no **conjunto universal** da Computação Quântica: **todas** as outras portas podem ser implementadas como a combinação de CNOTs e portas de qubit único.

Por ser equivalente a uma operação X controlada, a CNOT pode ser denominada CX, e é definida como a matriz:

$$CX = |0\rangle\langle 0| \otimes \mathbb{I}_Q + |1\rangle\langle 1| \otimes X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$CX = \frac{1}{2}(\mathbb{I} \otimes \mathbb{I} + \mathbb{I} \otimes X + Z \otimes \mathbb{I} - Z \otimes X)$$

Em código, além do método manual para se definir a porta lógica, podemos utilizar o método *cx*, que recebe como parâmetros o **índice** do qubit de controle (ou uma lista dos **controladores**) e o índice do qubit alvo (ou uma lista dos **alvos**).

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Graficamente, o operador pode ser representado normalmente como o círculo menor indicando o qubit de controle e a porta X no alvo, porém, no caso da CNOT há outra representação mais comum:

- A porta quântica X é equivalente à operação clássica XOR entre o estado do qubit com 1.
  - $X|0\rangle = |1\rangle$ é análogo a $0 \oplus 1 = 1$.
  - $X|1\rangle = |0\rangle$ é análogo a $1 \oplus 1 = 0$.
- Por consequência, CNOT equivale a um **XOR controlado**.
  - $CX|10\rangle = |1\rangle \otimes X|0\rangle = |11\rangle$.
  - Um XOR controlado seria $|1{,}0\rangle \rightarrow |1{,}0 \oplus 1\rangle = |11\rangle$.

Por causa disso, o alvo da porta CX é comumente representado por um $\oplus$ em vez da porta X, como mostra a figura “Portas CNOT”, que é o resultado de um circuito que aplica a porta CX duas vezes (a primeira utiliza o índice 0 como controle e 1 como alvo; na segunda, 1 é o controle e 0 o alvo).

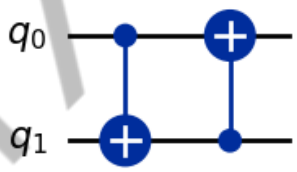


Figura 3 - Portas CNOT
Fonte: Elaborado pelo autor (2025)

3.4.2 Porta Toffoli

A operação de **Toffoli** é equivalente a uma **CNOT controlada**, uma operação NOT aplicada ao qubit alvo dependendo de **dois** qubits de controle. Por isso, também é denominada como porta **CCNOT** ou **CCX**.

É uma porta extremamente utilizada nos **algoritmos** mais comuns da Computação Quântica, também podendo servir como uma forma de se **sintetizar circuitos**, ou seja, decompor operações unitárias em outras mais simples.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Nota: sintetizar operações é um conceito fundamental para reduzir a complexidade de um circuito, garantindo que possa ser implementado em diferentes tipos de computadores quânticos (nem todo computador é capaz de realizar todas as operações unitárias) e diminuir as chances de ocorrer erros.

Matematicamente, a porta Toffoli é representada pela seguinte matriz:

$$CCX = |00\rangle\langle 00| \otimes \mathbb{I} + |01\rangle\langle 01| \otimes \mathbb{I} + |10\rangle\langle 10| \otimes \mathbb{I} + |11\rangle\langle 11| \otimes \mathbb{X} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Ao analisarmos a matriz como um somatório de produtos, é possível observar seu funcionamento: invertemos o qubit alvo **apenas** quando ambos qubits de controle estiverem no estado $|1\rangle$.

Além disso, podemos representar Toffoli com o **conjunto universal**, combinando portas CX com operações unitárias de um qubit. No caso, utilizaremos portas H e T.

Uma porta quântica que não foi mencionada é a porta de fase $P(\theta)$, que aplicam uma fase global ao estado (não alterando as probabilidades de medição), rotacionando o estado em um ângulo θ (em radianos). A porta T é um caso particular de P com $\theta = \pi/4$:

$$P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}; \;\; T = P(\pi/4) = \begin{pmatrix} 1 & 0 \\ 0 & \frac{1+i}{\sqrt{2}} \end{pmatrix}$$

O código-fonte "Circuito da porta Toffoli decomposta", cujo resultado está ilustrado na figura "Porta Toffoli decomposta", cria um circuito equivalente à porta CCX utilizando uma combinação de CX, H, T e $T^\dagger$. Vale destacar que, em Qiskit, é possível aplicar a porta T a um circuito com o método *t*, passando o índice do qubit como parâmetro, e a porta $T^\dagger$ com o método *tdg*, também passando o índice.

Tdg significa "*T dagger*", notação utilizada para representar $T^\dagger$ no circuito em alguns modos de visualização (como o modo *text*).

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

```
qc = QuantumCircuit(3)

qc.h(2)
qc.cx(1,2)
qc.tdg(2)
qc.cx(0,2)

qc.t(2)
qc.cx(1,2)
qc.tdg(2)
qc.cx(0,2)

qc.t([1,2])
qc.h(2)
qc.cx(0,1)

qc.t(0)
qc.tdg(1)
qc.cx(0,1)
```

Código-fonte 3 - Circuito da porta Toffoli decomposta
Fonte: Elaborado pelo autor (2025)

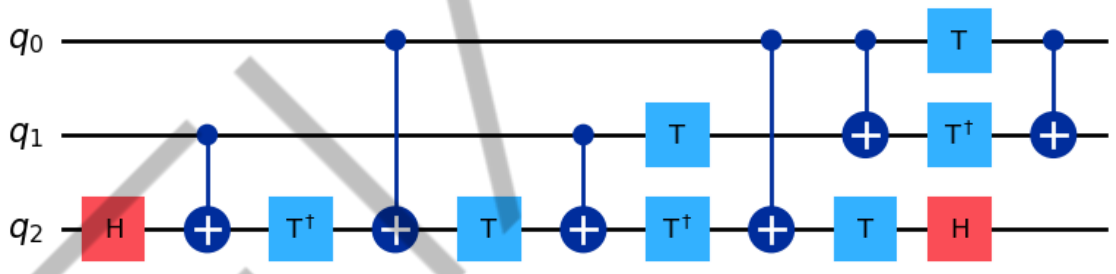


Figura 4 - Porta Toffoli decomposta
Fonte: Elaborado pelo autor (2025)

Nota: o circuito usa um par de Hadamards no qubit q_2, possuindo uma propriedade importante de interferência que será detalhada nas seções seguintes.

Assim como o alvo da porta CX pode ser descrito com o símbolo de XOR $\oplus$, o alvo da CCX também é representado com o mesmo símbolo, ilustrando que **mantém relação** com a operação clássica **XOR**.

Em Qiskit, também temos um método próprio para adicionar uma porta de Toffoli, *ccx*, que recebe como parâmetros o índice do primeiro qubit de controle (ou uma lista), o índice do segundo controle e o índice do alvo.

A figura a seguir mostra um circuito de 3 qubits com duas portas de Toffoli:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

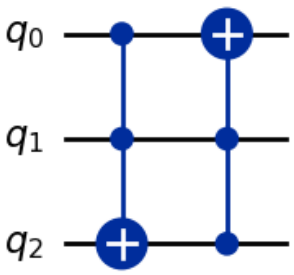


Figura 5 - Portas CCX
Fonte: Elaborado pelo autor (2025)

3.4.3 Porta SWAP

O operador **SWAP** é utilizado para trocar o estado de um par de qubits, de forma que $|ab\rangle \rightarrow |ba\rangle$. Essa porta lógica também é muito usada em **algoritmos**, podendo servir para **reorganizar** informação quântica ou **otimizar** e **sintetizar** circuitos. Matematicamente, representamos a $SWAP$ como:

$$SWAP = \frac{1}{2}(\mathbb{I} \otimes \mathbb{I} + X \otimes X + Y \otimes Y + Z \otimes Z) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

A decomposição em operadores de Pauli nos indica que a operação é **simétrica**, aplicando as mesmas portas simultaneamente em ambos os qubits. Essa característica é útil em técnicas de **correção de erros**, formas de **entrelaçamento** e **algoritmos**.

Decompor a porta $SWAP$ no conjunto universal é simples, necessitando de apenas três portas CX, como mostra o circuito do código-fonte:

```
qc = QuantumCircuit(2)
qc.cx([0,1,0], [1,0,1])
```

Código-fonte 4 - Circuito da porta SWAP decomposta
Fonte: Elaborado pelo autor (2025)

O resultado está na figura a seguir:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

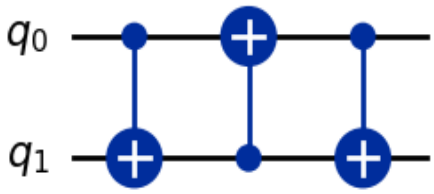


Figura 6 - Porta SWAP decomposta
Fonte: Elaborado pelo autor (2025)

Como a porta $SWAP$ **não tem direção**, ou seja, não importa a ordem na qual definimos o primeiro e segundo qubits que serão trocados, ela é representada por um "**X**" em cada qubit, interligados por uma linha, conforme ilustrado na figura "Porta SWAP".

Em Qiskit, a porta $SWAP$ pode ser adicionada por meio do método *swap*, o qual recebe como parâmetros os índices dos qubits (ou, como sempre, listas caso queira adicionar várias em sequência).

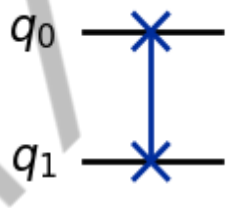


Figura 7 - Porta SWAP
Fonte: Elaborado pelo autor (2025)

3.4.4 Porta Fredkin

A porta **Fredkin**, também conhecido como **CSWAP**, é uma porta de 3 qubits equivalente a uma operação $SWAP$ controlada, trocando o estado de seus dois qubits alvo caso o único qubit de controle esteja em $|1\rangle$.

Assim como a $CNOT$, a porta $CSWAP$ é amplamente utilizada em algoritmos, para implementar operações **condicionais**, e em códigos de **correção de erros**, sendo importante para diminuir a complexidade de um circuito.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Representamos a porta de Fredkin como a matriz a seguir, considerando o qubit C como controle e o par A, B como alvos:

$$CSWAP = |0\rangle\langle 0|_C \otimes \mathbb{I}_A \otimes \mathbb{I}_B + |1\rangle\langle 1|_C \otimes SWAP_{A,B}$$

$$CSWAP = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Nota: para descrever $CSWAP$ em função apenas das matrizes de Pauli, basta substituir a porta $SWAP$ por sua representação e realizar os produtos.

Uma versão otimizada para a implementação da porta Fredkin utilizando apenas o conjunto universal foi proposta por Smolin e DiVincenzo (1996), por meio de uma versão controlada do seguinte operador V e seu dagger:

$$V = \frac{1+i}{2}\begin{pmatrix} 1 & -i \\ -i & 1 \end{pmatrix}; \quad V^\dagger = \frac{1-i}{2}\begin{pmatrix} 1 & i \\ i & 1 \end{pmatrix}$$

O código-fonte "Circuito da porta CSWAP decomposta" mostra uma implementação em Qiskit dos operadores V e $V^\dagger$, além do circuito proposto por Smolin e DiVincenzo, ilustrado na figura "Porta CSWAP decomposta". O código também demonstra como transformar uma matriz que representa o operador em uma porta lógica, utilizando o método *to_gate* — uma abordagem diferente daquela apresentada no início deste capítulo para a definição de operadores customizados.

```
from qiskit.quantum_info import Operator

COEF = (1+1j)/2

v_op = Operator([
    [COEF, (-1j)*COEF],
    [(-1j)*COEF, COEF],
])
v = v_op.to_instruction()
v.label="V"
cv = v.control(1)

v_dg_op = v_op.conjugate()
```

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

```
v_dg = v_dg_op.to_instruction()
v_dg.label = "V$^\dagger$"
# Utilize "Vdg" para renderizar em modos
# que não são "mpl"
cv_dg = v_dg.control(1)

qc = QuantumCircuit(3)

qc.cx(2,1)

qc.append(cv, [1,2])
qc.append(cv, [0,2])
qc.cx(0,1)
qc.append(cv_dg, [1,2])
qc.cx(0,1)

qc.cx(2,1)

qc.draw("mpl") # Figura 8
```

Código-fonte 5 - Circuito da porta CSWAP decomposta
Fonte: Elaborado pelo autor (2025)

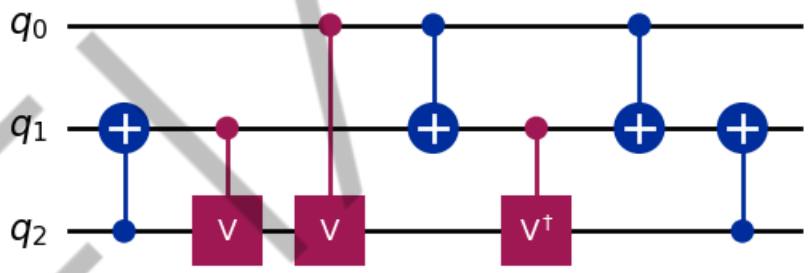


Figura 8 - Porta CSWAP decomposta
Fonte: Elaborado pelo autor (2025); adaptado de Smolin e DiVincenzo (1996)

No mesmo trabalho em que apresentam a decomposição, os autores também identificam que os operadores entre as portas $CNOT$ no início e no fim do circuito equivalem a uma porta Toffoli, o que permite reescrever o circuito conforme ilustrado na figura a seguir:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

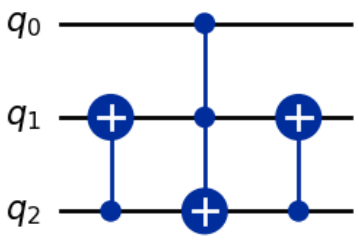


Figura 9 - Porta CSWAP decomposta com Toffoli
Fonte: Elaborado pelo autor (2025)

A representação “normal” da porta Fredkin é análoga à $SWAP$ com um qubit de controle, como ilustrado na figura a seguir. Em Qiskit, podemos adicionar a operação diretamente com o método *cswap*, que recebe como parâmetros o índice do qubit de controle e de ambos os alvos (ou listas).

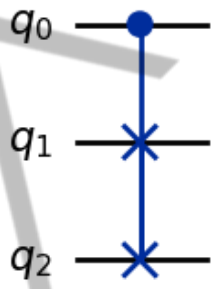


Figura 10 - Porta CSWAP
Fonte: Elaborado pelo autor (2025)

A porta Fredkin também é considerada a **operação universal** para implementar **circuitos clássicos reversíveis**.

Uma operação reversível é aquela em que podemos reconstruir os estados de entrada a partir do estado de saída.

Todas as operações **unitárias** (por consequência, qualquer porta quântica) são **reversíveis**, diferente das operações clássicas. Por exemplo, na operação clássica $A \oplus B = 1$, não é possível saber os estados de A e B apenas com o resultado do XOR.

Com a porta Fredkin, podemos construir **circuitos quânticos** que equivalem a uma versão **reversível** de um **circuito clássico**, propriedade que nos será útil para alguns algoritmos e modelos posteriormente.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

4 ESTADOS ENTRELAÇADOS

Anteriormente, vimos como construir estados compostos por meio do **produto tensorial** de n estados independentes, por exemplo: $|\psi\rangle = |a\rangle \otimes |b\rangle$. Esses estados são **separáveis**, pois podem ser decompostos nos estados individuais.

No entanto, existem estados multi-qubit que **não podem** ser descritos como um produto tensorial simples, como:

$$|\phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Esse estado está **normalizado** e **é válido,** mas é **impossível** encontrar estados $|a\rangle$ e $|b\rangle$ tais que $|a\rangle \otimes |b\rangle = |\phi\rangle$. Estados com essa propriedade são chamados de **emaranhados**, ou **entrelaçados**.

Em 1935, os cientistas Albert Einstein, Boris Podolsky e Nathan Rosen utilizaram um estado similar para argumentar que a teoria da mecânica quântica estava incompleta, o que ficou conhecido como **Paradoxo EPR**.

Os artigos de **John Stewart Bell** sobre o paradoxo, anos mais tarde, mostraram que as previsões da teoria quântica estavam corretas. Além disso, também demonstraram que o emaranhamento era uma propriedade fundamental da mecânica quântica, sem equivalente clássico, que poderia ter múltiplas aplicações, como veremos no decorrer do curso.

4.1 Paradoxo EPR

No início do desenvolvimento da mecânica quântica, muitos cientistas não acreditavam que a teoria estava correta por conta de sua natureza probabilística.

Em especial, o **Princípio da Incerteza de Heisenberg** levantava suspeitas. O Princípio diz que somos incapazes de determinar com precisão o momento e a posição de uma partícula quântica simultaneamente, levando a duas interpretações possíveis:

- Alguma falha no ferramental nos impede de realizar os cálculos (perturbações no estado durante a medição, erros nas equações etc.).

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

- Devido à natureza das partículas é, de fato, impossível calcular ambas as grandezas.

Embora atualmente se reconheça que a segunda interpretação é a correta, ela não é intuitiva — a ponto de ter sido inicialmente rejeitada até mesmo por Heisenberg. Uma das vertentes de cientistas acreditava que deveríamos considerar as **variáveis ocultas** nas equações da mecânica quântica para representar os dados ainda desconhecidos, mas que deveriam existir para que a mecânica quântica se tornasse determinística.

Para argumentar a favor da primeira interpretação do princípio da incerteza e da inclusão de variáveis ocultas, o trio EPR propôs o seguinte experimento mental:

Considere um par de partículas quânticas. Elas foram preparadas antes do experimento de forma que o estado de uma fosse **sempre o contrário** da outra.

Durante o preparo, as partículas são “conectadas”, de forma a criar o seguinte estado composto em superposição:

$$|\psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

Assim, sem quebrar a superposição, uma das partículas permanece no laboratório e a outra é enviada à Lua, criando uma grande **distância física** entre elas, mas **mantendo o estado composto** do sistema.

Aplicando os mesmos cálculos que vimos anteriormente, verificamos através das **amplitudes de probabilidade** que, ao medir o estado, ele pode colapsar em:

- $|01\rangle$, com 50% de chance.
- $|10\rangle$, com 50% de chance.

No entanto, como só temos **uma partícula** em nossa posse, o máximo que podemos fazer são **medições parciais**. Digamos que a nossa partícula é a “primeira” (“da esquerda”) e queremos calcular a probabilidade de medirmos o estado $|0\rangle$:

$$\begin{aligned}
(\Pi_0 \otimes \mathbb{I}_1)\,|\psi^-\rangle &= |0\rangle\langle 0| \otimes \mathbb{I}\frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \\
(\Pi_0 \otimes \mathbb{I}_1)\,|\psi^-\rangle &= \frac{1}{\sqrt{2}}(|0\rangle\,\langle 0|0\rangle\,|1\rangle - |0\rangle\,\langle 0|1\rangle\,|0\rangle) \\
(\Pi_0 \otimes \mathbb{I}_1)\,|\psi^-\rangle &= \frac{1}{\sqrt{2}}(|01\rangle - 0) = \frac{1}{\sqrt{2}}|01\rangle
\end{aligned}$$

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Com a amplitude de probabilidade calculada, é fácil ver que temos 50% de chance de medir o estado $|01\rangle$, que é a única possibilidade de resultado ao medir $|0\rangle$ no primeiro qubit. Isso está de acordo com o que já havíamos previsto; o mais interessante desse experimento são as **consequências da medição**.

Como fizemos uma medição parcial, precisamos re-normalizar o estado, dividindo a amplitude de probabilidade calculada por sua norma, que nesse caso será $\sqrt{\left|\frac{1}{\sqrt{2}}\right|^2} = \frac{1}{\sqrt{2}}$.

Assim, o estado resultante $|\psi^-{}_R\rangle$ será:

$$|\psi_R^-\rangle = \frac{\frac{1}{\sqrt{2}}|01\rangle}{\frac{1}{\sqrt{2}}} = |01\rangle$$

Existe 100% de chance de o estado colapsar em $|01\rangle$ após a medição parcial. Por mais que pareça um resultado óbvio ao analisar unicamente o vetor de estados (realmente só há uma possibilidade de estado quando o primeiro é zero), precisamos considerar suas implicações físicas.

As duas partículas estavam a **centenas de quilômetros** de distância; ao medirmos **apenas uma**, por consequência, o estado da outra **colapsou**, fornecendo informação sobre **todas as partículas do sistema**.

Nota: essa propriedade é conhecida como **não localidade**; estados emaranhados possuem correlação global, não importando a distância física entre eles.

Isso nos remete novamente às duas interpretações das leis da mecânica quântica: ou há alguma falha no nosso modelo — seja nos fundamentos matemáticos ou durante a medição —, ou o estado da partícula na Lua realmente colapsou por causa da medição realizada na Terra.

Einstein concluiu que a segunda opção implicaria na informação ser transmitida **instantaneamente** entre as partículas, o que violaria a **Teoria da Relatividade** (nada pode viajar mais rápido que a luz, nem mesmo informação).

Décadas depois da publicação original do Paradoxo EPR, após a morte de Einstein, o físico John Bell chegou a uma **inequação** que seria capaz de determinar

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

matematicamente a discussão quase filosófica sobre a natureza quântica, conhecida como **Desigualdade de Bell**. Infelizmente, os detalhes da demonstração e funcionamento da Desigualdade de Bell fogem do escopo deste curso.

O mais relevante para nosso estudo é compreender que, se os cientistas fizessem experimentos cujos resultados **satisfazem** a Desigualdade de Bell, a teoria quântica seria **incompleta** e a hipótese das **variáveis ocultas** estaria certa. No entanto, a teoria quântica previa casos que **nem sempre satisfazem** a inequação, ou seja, se os experimentos encontrassem tais resultados, a teoria **estaria correta** e a mecânica quântica realmente seria probabilística.

Desde a década de 80, cientistas efetuam tais experimentos na prática — inclusive, recebendo o prêmio Nobel de 2022, sempre com resultados que **não satisfazem** a Desigualdade de Bell. Portanto, a matemática **estava correta** e fenômenos como a **superposição** e **emaranhamento** existem e podem ser utilizados em problemas reais.

Os estudos de Bell e suas consequências demonstraram, ainda, que o emaranhamento é uma correlação de estados mais forte que qualquer estado clássico, fazendo com que o fenômeno servisse como base para o desenvolvimento da teoria da informação quântica.

4.2 Estados entrelaçados importantes

Retomando o que vinha sendo discutido, estados entrelaçados são aqueles que não podem ser descritos como um produto tensorial, por consequência, também não podem ser separados. Além disso, medir uma das partículas (ou qubits, no nosso caso) colapsa o estado, destruindo o emaranhamento.

Assim como existem infinitos estados compostos que podemos separar, podemos encontrar **incontáveis estados entrelaçados**. No entanto, alguns se destacam por sua importância, como os **quatro estados de Bell**, o **estado GHZ** e o **estado W**.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

4.2.1 Estados de Bell

Os **Estados de Bell**, também conhecidos como **Bases de Bell**, representam os estados emaranhados entre as **bases computacionais** de dois qubits. São eles:

$$\left|\Phi^{+}\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

$$\left|\Phi^{-}\right\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle)$$

$$\left|\Psi^{+}\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle)$$

$$\left|\Psi^{-}\right\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$$

Estes estados possuem o **maior grau de emaranhamento** dos estados de dois qubits: saber **qualquer** informação sobre um dos qubits automaticamente **colapsa o estado**, ao mesmo tempo em que aplicar **transformações unitárias locais** (portas lógicas aplicadas a um subconjunto dos qubits do sistema, mas não em todos ao mesmo tempo) **preserva** o grau de emaranhamento.

É possível gerar qualquer base de Bell com a relação disposta na equação a seguir, em que y' representa a negação de y.

$$|\beta_{xy}\rangle = \frac{1}{\sqrt{2}}(|0, y\rangle + (-1)^{x}|1, y'\rangle)$$

É fácil observar que:

$$|\beta_{00}\rangle = |\Phi^{+}\rangle; \quad |\beta_{10}\rangle = |\Phi^{-}\rangle$$

$$|\beta_{01}\rangle = |\Psi^{+}\rangle; \quad |\beta_{11}\rangle = |\Psi^{-}\rangle$$

Como consequência dessa equação e da propriedade de preservar o grau de emaranhamento, podemos descrever **todas as bases** em função da referência $|\Phi^{+}\rangle$:

$$|\Phi^{+}\rangle = (\mathbb{I} \otimes \mathbb{I})|\Phi^{+}\rangle$$

$$|\Phi^{-}\rangle = (\mathbb{I} \otimes Z)|\Phi^{+}\rangle$$

$$|\Psi^{+}\rangle = (\mathbb{I} \otimes X)|\Phi^{+}\rangle$$

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

$$|\Psi^-\rangle = (X \otimes Z)|\Phi^+\rangle$$

Os Estados de Bell são especialmente úteis em algoritmos e protocolos que se beneficiam desse forte entrelaçamento, como a **transmissão de estados** pela **teleportação quântica** (não é um teletransporte real, como na ficção), o protocolo de **codificação superdensa**, algoritmos de **criptografia**, testes da **não localidade quântica**, entre muitos outros.

4.2.2 Estado GHZ

Os estados de **Greenberger-Horne-Zeilinger**, conhecidos como **Estados GHZ**, são um tipo de emaranhamento caracterizado pela superposição simétrica entre os dois estados extremos, de forma geral:

$$|GHZ_N\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle^{\otimes N} + |1\rangle^{\otimes N}\right) = \frac{1}{\sqrt{2}}(|000\ldots0\rangle + |111\ldots1\rangle)$$

Com $N \geq 3$. A simbologia $|\psi\rangle^{\otimes N}$ indica um produto tensorial de $|\psi\rangle$ consigo mesmo N vezes. Por exemplo, o estado GHZ com 3 qubits é:

$$|GHZ_3\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle^{\otimes 3} + |1\rangle^{\otimes 3}\right) = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$$

Nota: isso não significa que podemos separar um estado GHZ em produtos tensoriais, é apenas uma forma compacta para se representar um estado genérico de N qubits, que só é possível por causa da repetição em cada estado.

Este estado pode ser entendido como uma **generalização** do emaranhamento observado nos Estados de Bell para **3 ou mais qubits**, sendo descrito pela primeira vez no **Paradoxo GHZ**, uma versão mais forte e complexa do paradoxo EPR que foge de nosso escopo, mas foi outro paradoxo que levantou discussões sobre a não localidade.

Como é possível observar pelo vetor de estado, os únicos resultados de medição são: **todos** os qubits colapsam para $|0\rangle$ ou todos para $|1\rangle$. Ou seja, medir **qualquer um** dos N qubits destrói a superposição de **todos** os qubits no emaranhamento **instantaneamente**.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Além da medição, caso haja algum erro ou interferência no circuito, o estado também é perdido e colapsado. Por isso, é considerado um emaranhamento frágil.

O estado foi utilizado para estudar o emaranhamento em sistemas maiores, e até hoje é amplamente utilizado em protocolos e algoritmos de **criptografia**, **comunicação segura**, **metrologia de alta precisão**, entre outros.

4.2.3 Estado W

O **Estado W** é um tipo de emaranhamento caracterizado por sua superposição simétrica: um qubit está no estado $|1\rangle$ e os demais estarão em $|0\rangle$, para todas as combinações possíveis, com $N \geq 3$.

$$|W_N\rangle = \frac{1}{\sqrt{N}}\left(\sum_{k=1}^{N} |0\rangle^{\otimes(k-1)} \otimes |1\rangle \otimes |0\rangle^{\otimes(N-k)}\right) = \frac{1}{\sqrt{N}}(|100\ldots0\rangle + |010\ldots0\rangle + |000\ldots1\rangle)$$

Nota: assim como os produtos tensoriais na formulação geral do estado GHZ, a fórmula acima não significa que o estado W pode ser separado em um produto.

Seu nome foi escolhido informalmente: em certas representações, como em diagramas de energia ou vetores de estado nas bases computacionais, alguns cientistas argumentaram que o estado lembrava a letra “W”. Um dos exemplos disso é o estado $|W_3\rangle$:

$$|W_3\rangle = \frac{|100\rangle + |010\rangle + |001\rangle}{\sqrt{3}}$$

O estado W possui um **emaranhamento distribuído**, correlacionando ***N-1*** qubits (todos exceto um) em vez de todos globalmente, como em GHZ. Como podemos observar por seu vetor de estados, se um qubit Q for medido como:

- $|0\rangle$, modificamos o emaranhamento de forma que o estado $Q = |1\rangle$ **deixa de existir**, porém, os outros estados **continuam emaranhados** e em superposição, formando o estado $|W_{N-1}\rangle$.
- $|1\rangle$, todos os outros colapsam para $|0\rangle$, **destruindo** o emaranhamento.

Por exemplo, ao realizarmos uma medição parcial em $|W_4\rangle$ para calcular a probabilidade de se medir o primeiro qubit como $|0\rangle$, teremos:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

$$P(|0\rangle_0) = ||0\rangle\langle 0| \otimes \mathbb{I} \otimes \mathbb{I}\,|W_4\rangle|^2 = \left|\frac{1}{\sqrt{4}}\right|^2 + \left|\frac{1}{\sqrt{4}}\right|^2 + \left|\frac{1}{\sqrt{4}}\right|^2 = \frac{3}{4}$$

O estado re-normalizado será:

$$|W_{4_R}\rangle = \frac{\frac{1}{2}(|0100\rangle + |0010\rangle + |0001\rangle)}{\sqrt{\frac{3}{4}}} = \frac{\frac{1}{2}(|0100\rangle + |0010\rangle + |0001\rangle)}{\sqrt{3}\frac{1}{2}} = \frac{(|0100\rangle + |0010\rangle + |0001\rangle)}{\sqrt{3}}$$

Isso mostra na prática que o estado $|1000\rangle$ **não faz mais parte** do emaranhamento após a medição.

Outra propriedade interessante surge quando simplificamos o estado re-normalizado. Como temos certeza que o primeiro qubit está no estado $|0\rangle$, podemos reescrever o emaranhamento omitindo-o:

$$|W_{4_R}\rangle = \frac{(|0100\rangle + |0010\rangle + |0001\rangle)}{\sqrt{3}} = \frac{(|100\rangle + |010\rangle + |001\rangle)}{\sqrt{3}} = |W_3\rangle$$

Ou seja, o estado resultante é **equivalente** ao estado W com um qubit a menos. Essa propriedade se mantém para **qualquer *N***, de forma que uma medição parcial de $|0\rangle$ em qualquer qubit no estado $|W_N\rangle$ resulta no estado $|W_{N-1}\rangle$. Vale destacar que, para $N = 3$, o estado resultante é $|\Psi^+\rangle$.

Por causa dessa propriedade, o estado W é mais **resiliente** que GHZ, representando um **emaranhamento forte**, que possui grandes chances de não colapsar mesmo após uma medição parcial.

Com essa robustez, o estado W é amplamente utilizado em sistemas que possuem altas chances de falhar, já que um erro aleatório não o afeta tanto quanto o estado GHZ. Alguns exemplos são protocolos de comunicação com transferência de estados, redes quânticas aplicadas a sistemas distribuídos, armazenamento de informação, entre outros algoritmos.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

5 CIRCUITOS QUÂNTICOS MULTI-QUBITS

Com base na descrição matemática apresentada na seção anterior e nas portas lógicas discutidas, é possível implementar circuitos capazes de gerar e manipular estados emaranhados, com destaque para as **Bases de Bell**, o **Estado GHZ** e o **Estado W**.

Além disso, abordaremos brevemente o conceito de **interferência**, transformando um **erro de fase** em um erro de **bit**, que será útil ao discutir técnicas de correção de erros.

5.1 Simulação com Qiskit Aer

Antes de prosseguir, vejamos outro uso do **Qiskit Aer**. No capítulo anterior, utilizamos a estrutura primitiva **Sampler** para gerar **amostras** de medição, que nos retornava as **probabilidades** para se medir cada estado possível, identificados por um **índice**.

Desta vez, utilizaremos um simulador de fato, que será executado múltiplas vezes, de forma semelhante ao Sampler, mas que retornará o **número de vezes** que cada resultado foi medido, além de identificar os estados por sua **representação em binário**.

Vamos repetir o exemplo do capítulo anterior, agora que sabemos trabalhar melhor com sistemas multi-qubits. Primeiramente, criaremos um circuito de **dois qubits** em que o **primeiro** terminará no estado **plus** e o **segundo** no **minus**:

$$(HX \otimes H)|00\rangle = |-+\rangle$$

Essa operação está anotada na **convenção do Qiskit**, ou seja, o “primeiro” qubit é o da **direita**; essa diferença entre a convenção usual e a do Qiskit será importante ao se trabalhar com o simulador.

Como os qubits não têm relação entre si, **podemos separar** o sistema em um produto tensorial. A partir dele, obtemos os possíveis estados de medição:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

$$|-+\rangle = |-\rangle \otimes |+\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$$

Também é fácil ver que cada estado tem 25% de probabilidade de ser medido.

O código-fonte "Simulação com Qiskit Aer" implementa as operações acima, adicionando medidores ao circuito. Após isso, ele **instancia o simulador** e executa **1000 medições**, cujos resultados são plotados em forma de um **histograma**, disposto na figura "Resultados da simulação". O código também define uma **função auxiliar** para resumir a medição, que será utilizada ao longo do capítulo.

Executamos o simulador AerSimulator de forma similar ao Sampler, instanciando a classe e invocando o método *run*, passando nosso circuito como parâmetro. Para o resultado, como não estamos calculando **probabilidades**, e sim **contagens**, invocamos os métodos *result* e *get_counts*.

```
from qiskit_aer import AerSimulator
from qiskit.result.counts import Counts
from qiskit.visualization import plot_histogram

def measure(qc: QuantumCircuit) -> Counts:
    backend = AerSimulator()
    job = backend.run(qc, shots=1000)
    return job.result().get_counts()


qc = QuantumCircuit(2)
qc.x(1)
qc.h([0,1])
qc.measure_all()

results = measure(qc)
plot_histogram(results) # Figura 11
```

Código-fonte 6 - Simulação com Qiskit Aer
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

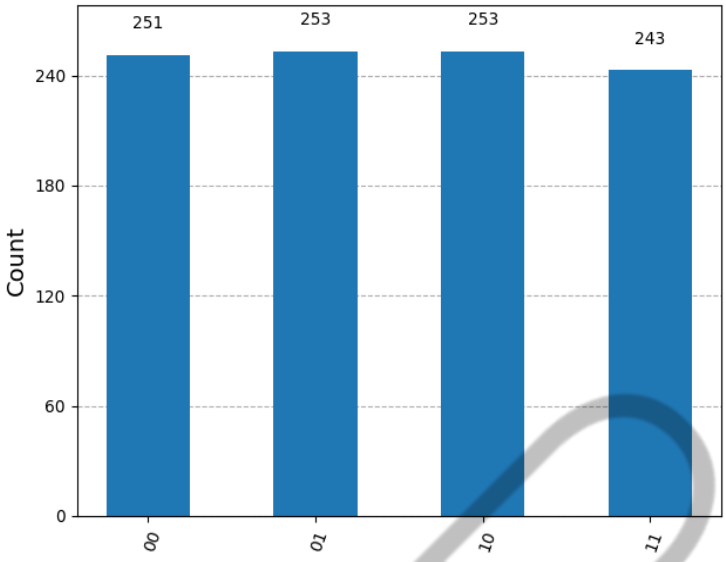


Figura 11 - Resultados da simulação
Fonte: Elaborado pelo autor (2025)

Como executamos mil vezes, os resultados **são coerentes** com o esperado, já que cada estado foi medido cerca de 250 vezes. O número de execuções é **arbitrário**, sendo que **mais medições** resultam em uma estimativa **mais precisa** — ainda que isso possa levar mais tempo.

5.2 Interferência e Circuito HZH

Ao final do capítulo, abordaremos o conceito de **correção de erros**, porém, antes que seja possível corrigir, precisamos de uma ferramenta para **detectar** erros.

Um dos principais tipos de erro que pode ocorrer no sistema é uma **inversão de fase**, que pode ser **simulada** em um circuito através da porta Z, por ter uma ação análoga.

Como a inversão de fase **não altera** o resultado da medição, é difícil detectá-lo em circuitos simples. Isso **não significa** que os erros podem ser ignorados, visto que ainda conseguem **alterar estados** mais complexos de formas que não queremos.

Assim, para que seja possível detectar uma inversão de fase, podemos utilizar o conceito de **interferência**.

Resumindo o que vem sendo estudado desde o início do curso: partículas quânticas possuem ao mesmo tempo propriedades de **partículas** e de **ondas**.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Enquanto estão em superposição, as partículas não possuem posição específica, mas sim uma **função de onda** que utilizamos para calcular suas **amplitudes de probabilidade**. A aplicação de um operador à partícula causa uma **interferência construtiva** ou **destrutiva** em sua função de onda. Aplicar um operador unitário duas vezes em sequência causa uma **interferência destrutiva**, anulando seu efeito (o que explica fisicamente a propriedade $UU = \mathbb{I}$).

No caso da porta H, aplicá-la uma vez **cria** uma superposição, enquanto aplicá-la duas vezes **desfaz** a superposição devido à interferência. Se aplicarmos outros operadores **entre** as portas Hadamard, conseguimos manipular o estado superposto **antes** da interferência, **alterando** o estado resultante.

Esse é o princípio que usaremos para detectar erros de fase: forçamos uma interferência utilizando um par de Hadamards para **manter** o estado do sistema caso nada inesperado ocorra. No entanto, uma **inversão de fase** será transformada em **inversão de qubit**, visto que:

$$HH|0\rangle = H|+\rangle = H|0\rangle; \quad HH|1\rangle = H|-\rangle = H|1\rangle$$

$$HZH|0\rangle = H|-\rangle = H|1\rangle; \quad HZH|1\rangle = H|+\rangle = H|0\rangle$$

Dessa forma, o erro anteriormente **indetectável** pela medição se torna **evidente**.

O próximo código-fonte cria dois circuitos: o primeiro aplica uma porta Z no estado $|1\rangle$ (criado ao aplicar uma porta X no circuito inicial), simulando uma inversão de fase não detectada. O segundo circuito replica o primeiro, mas criando uma interferência com um par de H (*após* a aplicação de X). Para demonstrar os efeitos da interferência, ambos são medidos. Os resultados são mostrados na tela com a função *display* do Jupyter, indicados no código como um comentário.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

```
qc = QuantumCircuit(1)
qc.x(0)
qc.z(0) # Erro de inversão de fase
qc.measure_all()

qc2 = QuantumCircuit(1)
qc2.x(0)
qc2.h(0)
qc2.z(0) # Erro de inversão de fase
qc2.h(0)
qc2.measure_all()

display(measure(qc))  # {'1': 1000}
display(measure(qc2)) # {'0': 1000}
```

Código-fonte 7 - Identificando inversão de fase com interferência
Fonte: Elaborado pelo autor (2025)

No primeiro circuito, o resultado foi $|1\rangle$ em todas as medições, conforme previsto. Embora saibamos que houve um erro — introduzido propositalmente —, em um circuito real esse erro poderia ter passado **despercebido**. Já os resultados do segundo circuito indicam uma inversão no estado esperado do qubit, **evidenciando o erro**, apesar de termos um resultado indesejado.

Nota: não devemos adicionar outras portas entre os Hadamards, sob o risco de alterar o estado em superposição, causando efeitos indesejados após a interferência.

Manipular interferência é uma técnica utilizada não apenas na **detecção de erros**, mas também para **aumentar a resiliência** de estados emaranhados e de **operadores** que podem ser utilizados para gerá-los, como foi visto na **decomposição** da porta **Toffoli**.

5.3 Circuitos de estados emaranhados

Com as considerações anteriores feitas, finalmente podemos implementar circuitos que geram os estados emaranhados em destaque.

Para cada estado, descreveremos como **montar o circuito**, desde a preparação dos estados iniciais até a discussão das portas que devem ser aplicadas.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

5.3.1 Estados de Bell

Como visto anteriormente, podemos gerar todas as Bases de Bell a partir do estado $|\Phi^+\rangle$, tornando simples implementar qualquer outro estado uma vez que tenhamos este primeiro.

Para preparar o estado $|\Phi^+\rangle$, é preciso que os qubits entrem em uma superposição de forma que sempre tenham o mesmo estado. Isso pode ser descrito como uma operação condicional:

- Se o primeiro qubit estiver em $|0\rangle$, o segundo deve ser $|0\rangle$. O resultado será o estado $|00\rangle$.
- Se o primeiro for $|1\rangle$, o segundo deve ser $|1\rangle$. O resultado será o estado $|11\rangle$.

Implementar cada um dos casos isoladamente não é difícil em Qiskit. Como todos os qubits iniciam no estado $|0\rangle$, basta **não fazer nada** que o estado resultante será $|00\rangle$, cumprindo o primeiro caso. Da mesma forma, **aplicar *X*** em ambos qubits resulta em $|11\rangle$, satisfazendo o segundo caso. Agora, precisamos de uma forma de **unir** os dois casos, fazendo com que cada um tenha 50% de chances de ser medido.

Como mencionado anteriormente, o **operador Feynman** é utilizado para **implementar condições**. Considerando um circuito no estado inicial e aplicando CX com o primeiro qubit como controle e o segundo como alvo, temos que:

- Se o controlador estiver em $|0\rangle$, nada acontecerá no alvo. O resultado será o estado $|00\rangle$.
- Se o controlador for $|1\rangle$, a porta aplicará X no alvo. O resultado será o estado $|11\rangle$.

Com isso, **relacionamos** os dois qubits, mas ainda não estão emaranhados, já que só há uma possibilidade por vez. No entanto, ao aplicarmos uma porta **Hadamard** no qubit controlador antes da CX, teremos a seguinte relação:

- O qubit de controle estará no estado $|+\rangle$. A superposição indica que o qubit está, ao mesmo tempo, nos estados $|0\rangle$ e $|1\rangle$.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

- Após a medição, o controlador terá 50% de chance de estar em $|0\rangle$, onde nada ocorre no alvo, resultando em $|00\rangle$.
- Também terá 50% de chance de ser medido como $|1\rangle$, caso no qual o alvo é negado, resultando em $|11\rangle$.

Dessa forma, a porta CX fez com que o qubit alvo **também entrasse em superposição**, de forma que ambos **terão o mesmo estado** após uma medição. Saber o estado de um dos qubits nos faz **descobrir o estado** do outro, ou seja, nosso sistema está **emaranhado**, de forma que representa o estado $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, ou seja, $|\Phi^+\rangle$.

O código-fonte implementa o procedimento descrito, medindo o circuito ao final:

```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0,1)
qc.measure([0,1], [0,1])

display(qc.draw("mpl")) # Figura 12
display(measure(qc))    # {'00': 519, '11': 481}
```

Código-fonte 8 - Circuito Phi plus
Fonte: Elaborado pelo autor (2025)

A figura mostra a versão final do circuito:

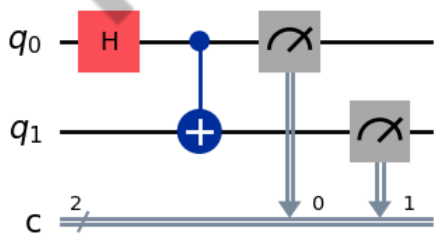


Figura 12 - Circuito Phi plus
Fonte: Elaborado pelo autor (2025)

Para os outros três estados de Bell, basta aplicar as relações descritas, sempre tomando cuidado com a diferença entre as convenções usuais e as convenções do Qiskit. A próxima figura mostra a implementação em circuito dos quatro estados, utilizando $|\Phi^+\rangle$ como base. Respectivamente, temos $|\Phi^+\rangle$ no quadrante superior esquerdo, $|\Phi^-\rangle$ no superior direito, $|\Psi^+\rangle$ no inferior esquerdo e $|\Psi^-\rangle$ no inferior direito.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

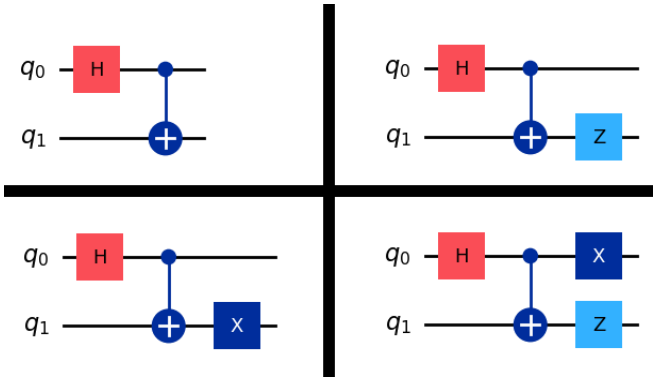


Figura 13 - Circuitos para as Bases de Bell
Fonte: Elaborado pelo autor (2025)

Os códigos são simples de se implementar, e as medições retornam o esperado, cerca de 50% cada estado possível, que são $|00\rangle$ e $|11\rangle$ nos estados Φ ou $|01\rangle$ e $|10\rangle$ nos estados Ψ.

5.3.2 Estado GHZ

A lógica para criar qualquer variação do estado GHZ é **similar ao estado** $|\Phi^+\rangle$. Nos estados GHZ, apesar de termos um emaranhamento de N qubits, todos deverão ter o mesmo valor, resultando em dois estados com 50% de probabilidade de medição cada, da mesma forma que $|\Phi^+\rangle$, que é análogo a um **estado GHZ de 2 qubits**.

Vamos começar do mesmo modo que o circuito $|\Phi^+\rangle$: um sistema $|\psi\rangle$ com N qubits no estado $|0\rangle$. Ao aplicar **Hadamard** no primeiro qubit, que será nosso controlador, chegaremos no estado intermediário $|\psi_1\rangle$ (utilizando as convenções usuais):

$$|\psi_1\rangle = (H \otimes \mathbb{I} \otimes \mathbb{I} \otimes \cdots \otimes \mathbb{I})|000\cdots\rangle = \frac{1}{\sqrt{2}}|+\rangle \otimes |000\cdots\rangle$$

Ao aplicarmos uma porta **Feynman** entre o controlador e o qubit seguinte, criamos um **emaranhamento** entre ambos, equivalente ao estado $|\Phi^+\rangle$. Não é difícil perceber que, ao aplicar outra CX entre o **controlador** e o **terceiro** qubit, criaremos um emaranhamento entre os três qubits utilizados até então.

Podemos provar por indução que **aplicar CNOTs em sequência**, entre o qubit alvo e **cada um** dos subsequentes, até o N-ésimo, resultará no estado GHZ para N qubits.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

O código-fonte a seguir exemplifica a criação de um estado GHZ para $N = 5$, ilustrado na figura "Circuito GHZ de 5 qubits". A medição demonstra os resultados esperados: cerca de 50% para o estado em que todos os qubits são medidos como $|0\rangle$ e cerca de 50% para o estado no qual todos são $|1\rangle$.

```
N = 5

qc = QuantumCircuit(N)
qc.h(0)
qc.cx(0, [i for i in range(1,N)])
qc.measure_all()

display(qc.draw("mpl")) # Figura 14
display(measure(qc)) # {'11111': 505, '00000': 495}
```

Código-fonte 9 - Circuito GHZ de 5 qubits
Fonte: Elaborado pelo autor (2025)

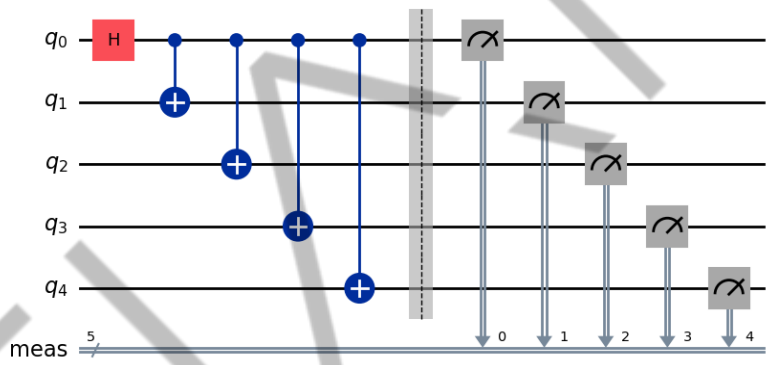


Figura 14 - Circuito GHZ de 5 qubits
Fonte: Elaborado pelo autor (2025)

Esta implementação é genérica: substituir o valor da constante para qualquer $N \geq 2$ resultará no circuito que gera o estado $|GHZ_N\rangle$. Para N grandes, outras implementações podem ser mais performáticas, como variações utilizando portas Toffoli.

5.3.3 Estado W

Apesar de não parecer à primeira vista, o estado $|W_N\rangle$ é **similar** ao $|GHZ_{N1}\rangle$, diferenciados pela **fase** de sua superposição: os estados GHZ criam uma superposição que sempre representa uma **polarização de 50%**; já os estados W precisam de uma **superposição controlada**, para que polarize em $\frac{1}{N}$.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

O maior desafio ao tentar implementar o circuito é preparar o estado com essa superposição.

Existem várias implementações para os estados W, otimizando para um N específico ou focando na simplicidade. O circuito que construiremos é uma **variação genérica** para N qubits, não sendo considerado a melhor opção em nenhum quesito, mas simples o bastante para que possamos entender como criar a superposição.

A implementação descrita é uma adaptação do código feito por Pierre Decoodt (2018), submetido ao evento "*Teach Me QISKit Award 2018*".

Para criar a superposição, o artigo original define um operador $F_{k,k+1}$ que relaciona **dois qubit** com uma porta CZ e aplica **rotações** em torno do eixo y, utilizando o ângulo θ:

$$\theta_k = arccos\left(\frac{1}{\sqrt{N-k+1}}\right)$$

Em que k é o índice do qubit **controlador**, $k+1$ é o índice do qubit **alvo** e N é o **número** de qubits. A porta F é equivalente a aplicar as seguintes operações:

- Uma rotação $R_y(-\theta_k)$ aplicado ao qubit q_{k+1}.
- Uma operação CZ entre os qubits q_k e q_{k+1}. A direção não importa, mas convencionamos q_k como controlador.
- Outra rotação, $R_y(\theta_k)$ aplicado a q_k.

Antes de aplicar F, preparamos o estado de forma a obter o estado inicial $|\phi_0\rangle = |100\cdots 0\rangle$, na **convenção usual**.

Assim, aplicamos as portas F ***N-1*** vezes **em sequência**, resultando no emaranhamento:

$$|\phi_1\rangle = F_{N-1,N} \cdots F_{k,k+1} \cdots F_{2,3}\, F_{1,2}|\phi_0\rangle$$

$$|\phi_1\rangle = \frac{1}{\sqrt{N}}(|10\cdots 0\rangle + |11\cdots 0\rangle + \cdots + |11\cdots 1\rangle)$$

Por fim, ajeitamos nosso estado ao aplicar ***N-1*** portas $CX_{k+1,k}$, utilizando o qubit k como controlador e $k+1$ como alvo, resultando no estado $|W_N\rangle$. É importante que

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

sejam aplicadas dos qubits mais “à esquerda” para os “à direita” na **convenção usual** para formar o estado correto.

$$CX_{N,N-1}\, CX_{N-1,N-2} \cdots CX_{k,k-1} \cdots CX_{2,1}|\phi_1\rangle = |W_N\rangle$$

O código-fonte “Circuito W de 4 qubits” cria uma função que calcula o ângulo θ_k e aplica as operações equivalente à porta F no circuito, sem criar um operador customizado pela simplicidade. A seguir, ele implementa o estado de forma genérica, desenhando o circuito para $N = 4$ como exemplo.

Atenção: cuidado com a diferença entre a convenção usual e do Qiskit para se representar estados.

```
def apply_f(
    qc: QuantumCircuit,
    n: int,
    k: int
) -> None:
    target = n - k
    control = target - 1

    theta = np.arccos(np.sqrt(1/(n-k+1)))
    qc.ry(-theta, control)
    qc.cz(target, control)
    qc.ry(theta, control)


N = 3
qc = QuantumCircuit(N)
qc.x(-1)

for k in range(1, N):
    apply_f(qc, N, k)

for i in reversed(range(N-1)):
    qc.cx(i, i+1)

qc.measure_all()
display(qc.draw("mpl"))      # Figura 15
plot_histogram(measure(qc)) # Figura 16
```

Código-fonte 10 - Circuito W de 4 qubits
Fonte: Elaborado pelo autor (2025)

A figura “Circuito W de 4 qubits” ilustra o circuito gerado, enquanto a figura “Medições do Circuito W de 4 qubits” mostra o histograma com os resultados de sua medição.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

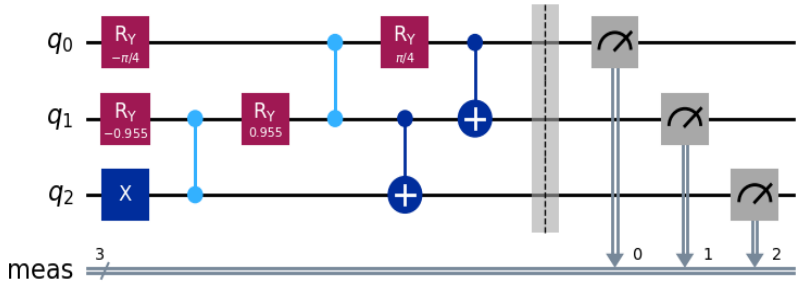


Figura 15 - Circuito W de 4 qubits
Fonte: Elaborado pelo autor (2025)

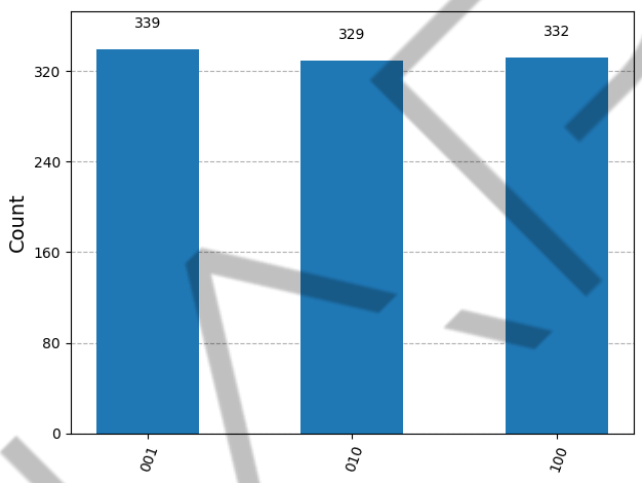


Figura 16 - Medições do Circuito W de 4 qubits
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

6 INTRODUÇÃO À CORREÇÃO DE ERROS QUÂNTICOS

A partir do próximo capítulo, abordaremos o desenvolvimento de **algoritmos**, **modelos** e outras **aplicações práticas** da Computação Quântica, sempre utilizando simuladores ou estruturas primitivas quando for necessário realizar medições.

Ainda assim, é importante destacar que, no mundo real, a criação de circuitos **não é tão simples**. Para que a Computação Quântica seja útil, precisamos **isolar o sistema** do ambiente, ao mesmo tempo em que **permitimos o controle** do mesmo por agentes externos. No entanto, os estados quânticos são extremamente frágeis, colapsando com a menor das variações no ambiente e sofrendo erros aleatórios devido a inúmeros fatores incontroláveis, entre outros.

Por isso, a **Correção de Erros Quânticos** é uma das áreas mais estudadas atualmente, e uma das mais necessárias. A seguir, abordaremos os dois tipos mais básicos de erro: **inversão de bits** (*bit-flip*) e a **inversão de fase** (*phase-flip*). Também veremos o **Código de Shor**, uma alteração no circuito que identifica e corrige os erros, contanto que sejam poucos.

Nota: por serem análogos às portas X e Z de Pauli, esses erros são conhecidos como erros de Pauli.

6.1 Simulação de erros com Qiskit Aer

Uma das vantagens de se utilizar o simulador do **Qiskit Aer** é que ele nos permite criar um **modelo com ruído**, simulando os efeitos de **interferência** em um circuito real que causam erros de inversão. Para isso, basta instanciar a classe *NoiseModel* e passá-la como parâmetro no *job* do simulador.

O próximo código-fonte exemplifica a simulação de um **erro de Pauli**. Primeiro, cria o estado $|GHZ_3\rangle$, que tem apenas dois resultados possíveis. Então, introduz a possibilidade de ocorrer uma **inversão de bit** após cada porta lógica com 10% de chance.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

```
from qiskit_aer.noise import NoiseModel, pauli_error

qc = QuantumCircuit(3)
qc.h(0)
qc.cx([0,0],[1,2])
qc.measure_all()

# 10% de aplicar X, 90% de não acontecer nada
bit_flip_error = pauli_error([("X", 0.1), ("I", 0.9)])

# Erros em portas de N qubits devem afetar N qubits
# Mesmo bit flip, adaptado para 2 qubits:
cx_error = bit_flip_error.tensor(bit_flip_error)

noise_model = NoiseModel()
noise_model.add_all_qubit_quantum_error(
    bit_flip_error, ["h"]
)
noise_model.add_all_qubit_quantum_error(
    cx_error, ["cx"]
)

job = AerSimulator().run(
    qc, shots=1000, noise_model=noise_model
)
result = job.result().get_counts()
plot_histogram(result) # Figura 17
```

Código-fonte 11 - Circuito GHZ com erros
Fonte: Elaborado pelo autor (2025)

A figura mostra os resultados de medição, evidenciando o erro:

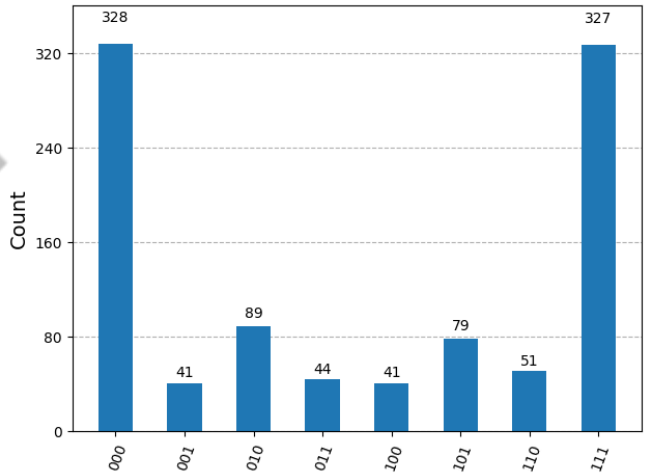


Figura 17 - Medições do Circuito GHZ com erros
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Apesar dos resultados corretos serem mais prováveis, a quantidade de novos estados introduzidos mostra que não podemos ignorar os erros. Simuladores como o Aer ajudam a testar como o sistema se comporta em cenários realistas, possibilitando minimizar erros antes de executar em um computador quântico.

6.2 Princípios da correção de erros

Para corrigir erros quânticos, precisamos ter em mente dois fenômenos que limitam nossa capacidade de proteger informação.

O **Teorema da Não Clonagem** diz que é impossível criar cópias idênticas de um estado desconhecido. A principal implicação é que não podemos replicar qubits diretamente para criar redundância, como fazemos em sistemas clássicos. Formalmente, não existe operador U tal que:

$$U(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |\psi\rangle$$

Outra limitação é a necessidade de **medições indiretas** dos qubits. Como medir diretamente **destruiria superposições**, precisamos introduzir **qubits auxiliares** ao sistema, chamados **ancillas**, para que possamos realizar medições sem colapsar o sistema principal. As medições indiretas detectam **padrões** capazes de **identificar erros**, chamados de **síndromes**.

6.2.1 Código de Repetição e Bit-Flip

O **Código de Repetição** é o equivalente quântico mais simples da redundância para proteção contra erros. Ele contorna o Teorema da Não Clonagem ao combinar um emaranhamento controlado com medição indireta de síndromes, sendo capaz de detectar erros sem perturbar a informação principal. O algoritmo codifica a informação através do mapeamento:

$$|0\rangle \rightarrow |000\rangle\,; \quad |1\rangle \rightarrow |111\rangle$$

É possível repetir o estado **quantas vezes quisermos**, mas as versões mais básicas utilizam esse mapeamento no qual cada qubit do sistema principal será repetido **três vezes**.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

Para implementar, começamos pela etapa de **codificação** (encoding): introduzimos $N-1$ **qubits extra** no sistema. Então, criamos um **emaranhamento** entre o qubit original e os auxiliares, formando um **estado GHZ** de ***N*** qubits.

Supondo que o único erro possível é o bit-flip, seguimos para a etapa de detecção de erros: criamos ancillas para comparar os valores do qubit original com os auxiliares. No caso do enconding de 3 qubits, precisamos de duas ancillas, uma comparando o qubit original q_0 com o auxiliar q_1, outro para comparar q_0 e q_2.

A figura a seguir mostra um circuito que implementa o código, codificando q_0 e medindo sua síndrome. Não importa o valor original de q_0 para a síndrome, já que se não ocorrer um erro o circuito aplicará um par de X cada ancilla, medindo 0.

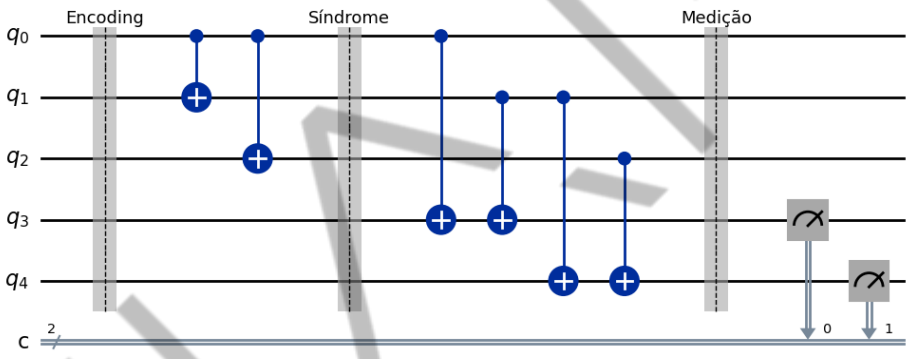


Figura 18 - Código de repetição para detecção de bit-flip
Fonte: Elaborado pelo autor (2025)

Caso ocorra um erro, a síndrome indica em qual qubit ele se manifestou, permitindo que seja corrigido posteriormente, como ilustra o quadro:

Estado	Síndrome	Correção
$\alpha\|000\rangle+\beta\|111\rangle$	00	$\mathbb{I}\otimes\mathbb{I}\otimes\mathbb{I}$
$\alpha\|100\rangle+\beta\|011\rangle$	10	$X\otimes\mathbb{I}\otimes\mathbb{I}$
$\alpha\|010\rangle+\beta\|101\rangle$	11	$\mathbb{I}\otimes X\otimes\mathbb{I}$
$\alpha\|001\rangle+\beta\|110\rangle$	01	$\mathbb{I}\otimes\mathbb{I}\otimes X$

Quadro 1 - Síndromes para bit-flip
Fonte: Elaborado pelo autor (2025)

O conceito se aplica para **qualquer estado** inicial de q_0, mesmo que seja **desconhecido**. A desvantagem desse método é que, além de apenas detectar erros de bit-flip, só é capaz de **detectar um erro**. Caso dois erros aconteçam ao mesmo tempo, não será possível identificar pelas síndromes.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

6.2.2 Adaptando para corrigir Phase-Flip

O código de repetição não corrige erros de **phase-flip**; inclusive, por ter mais portas, ele **aumenta** as chances de ocorrer. Como vimos anteriormente, é **impossível** detectar esses erros ao medir nas bases computacionais, mas as **bases de Hadamard** ($|+\rangle$ e $|-\rangle$) são capazes.

Dessa forma, **adaptaremos** nosso encoding para o seguinte mapeamento, utilizando portas Hadamard:

$$|0\rangle \rightarrow |+++\rangle\,; \qquad |1\rangle \rightarrow |---\rangle$$

Após a medição da síndrome, adicionaremos mais portas H em cada qubit para que a **interferência** destrua a superposição criada, como ilustra a figura:

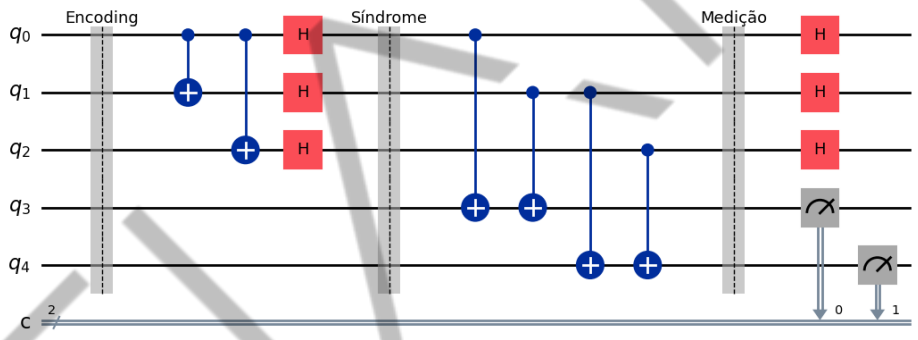


Figura 19 - Código de repetição para detecção de phase-flip
Fonte: Elaborado pelo autor (2025)

É possível **simplificar** o circuito:

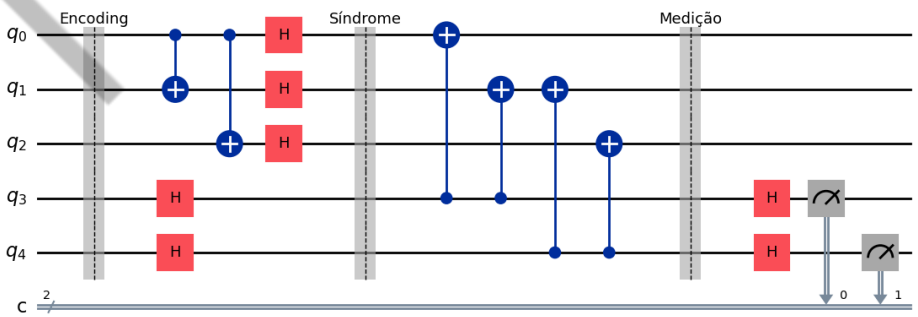


Figura 20 - Código para detecção de phase-flip adaptado
Fonte: Elaborado pelo autor (2025)

O código adaptado possui a limitação de ser capaz de detectar inversões de fase, mas perde a habilidade de corrigir bit-flips.

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

O quadro a seguir mostra as novas síndromes acusadas pelo código:

Estado	Síndrome	Correção
$\alpha\|+++\rangle+\beta\|---\rangle$	00	$\mathbb{I}\otimes\mathbb{I}\otimes\mathbb{I}$
$\alpha\|-++\rangle+\beta\|+--\rangle$	10	$Z\otimes\mathbb{I}\otimes\mathbb{I}$
$\alpha\|+-+\rangle+\beta\|-+-\rangle$	11	$\mathbb{I}\otimes Z\otimes\mathbb{I}$
$\alpha\|++-\rangle+\beta\|--+\rangle$	01	$\mathbb{I}\otimes\mathbb{I}\otimes Z$

Quadro 2 - Síndromes para phase-flip
Fonte: Elaborado pelo autor (2025)

6.3 Código de Shor

O Código de Shor de 9-qubits, proposto por Peter Shor em 1995, foi a primeira forma prática para se corrigir erros de bit e phase-flip simultaneamente, combinando os dois códigos de repetição abordados.

O encoding codifica o qubit original (lógico) em 9 qubits (físicos), organizados em três blocos de 3 qubits. Cada bloco utiliza nosso primeiro código de repetição para se proteger contra bit-flips (código interno); emaranhamos os 3 blocos na base de Hadamard para proteger contra phase-flips (código externo). O novo mapeamento será:

$$|0\rangle \rightarrow \frac{1}{\sqrt{8}}(|000\rangle+|111\rangle)^{\otimes 3}\ ; \quad |1\rangle \rightarrow \frac{1}{\sqrt{8}}(|000\rangle-|111\rangle)^{\otimes 3}$$

Poderíamos aplicar os códigos interno e externo em qualquer ordem, mas iniciaremos pelo externo, emaranhando o qubit lógico com os dois primeiros auxiliares e, depois, implementando o código interno nos blocos. A detecção de síndrome também é similar ao código que corrige erros de fase.

A figura a seguir mostra uma implementação do Código de Shor:

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

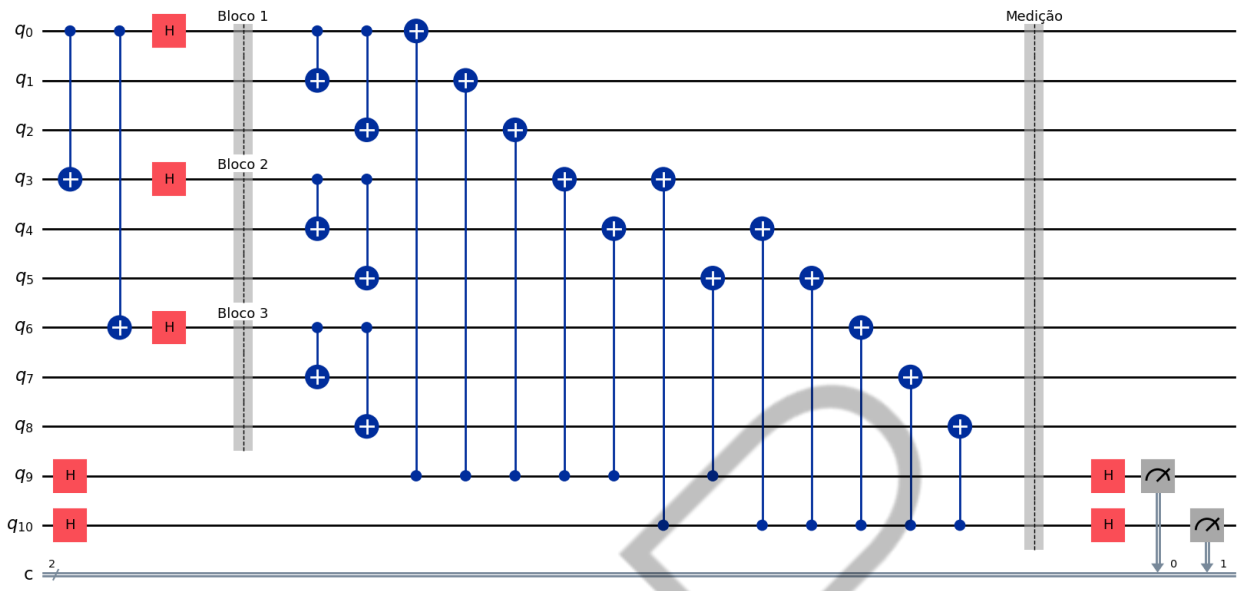


Figura 21 - Código de Shor
Fonte: Elaborado pelo autor (2025)

Cada código interno consegue corrigir **até um** erro bit-flip ao analisar a síndrome. O código externo sozinho faria o mesmo para erros de fase, mas o código interno atrapalha. As portas $CNOT$ após a repetição é uma versão simplificada de um código que **decodifica** o código interno, utiliza o externo com os 6 qubit internos para **detectar phase-flip**, **re-codifica** o código interno e, por fim, podemos corrigir o erro detectado.

Com o circuito, é possível **detectar** e **corrigir** erros de **bit** e **phase-flip** simultaneamente, contanto que seja **um erro arbitrário** (bit ou phase) em algum um dos **9 qubits**, ou um **erro combinado** (bit e phase-flip no mesmo qubit) em apenas um qubit.

Ele não é capaz de corrigir dois ou mais erros em qubits diferentes.

Outro detalhe a ser considerado é que, em qualquer circuito, a probabilidade de ocorrência de erros é **diretamente proporcional** à quantidade de portas lógicas. Isso significa que nosso código ainda pode **aumentar** a probabilidade de erros, uma vez que introduzimos ao menos 23 portas novas no circuito original.

Ao comparar a probabilidade de ocorrência de erros no sistema original, p, com a probabilidade de erros com o código de Shor, dada por $1-(1-p)^9-9p(1-p)^8$, observamos que o ***break-even*** ocorre pouco acima dos **3%**. Em outras palavras, o

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

código de Shor só melhora nosso circuito, corrigindo erros, se a probabilidade de ocorrer um erro for **menor que 3%**, o que é muito baixo.

Estudamos o código de Shor por motivos **históricos**, já que foi o primeiro a conseguir corrigir os dois tipos de erro com o mesmo algoritmo e aplicar diretamente **técnicas básicas** para a correção de erros, servindo como uma boa **introdução** ao assunto, mas ele não é utilizado no mundo real por já existirem alternativas **mais eficientes**.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

CONCLUSÃO

Ao longo do capítulo, estendemos o que foi visto anteriormente para sistemas compostos por múltiplos qubits, que podem ser separáveis ou estar em um estado emaranhado. Demonstramos os principais estados emaranhados: as Bases de Bell, Estados GHZ e Estados W, listando algumas de suas utilidades e propriedades.

Apresentamos operadores que agem tanto de forma local nos qubits de sistemas compostos quanto de forma global no sistema como um todo, destacando algumas portas multi-qubits, como a porta CNOT, Toffoli, SWAP e Fredkin. Com simulações do Qiskit, demonstramos como utilizar essas portas para construir e medir circuitos que produzem os estados emaranhados mencionados.

Também discutimos a interferência e como pode ser utilizada para aumentar a tolerância a erros de circuitos. Por fim, realizamos uma introdução à correção de erros quânticos, ilustrando os princípios da área com o Código de Shor, que apesar de pouco útil na prática é importante historicamente.

Esses conceitos formam a base mínima necessária para implementarmos qualquer algoritmo mais complexo, incluindo algoritmos de busca e ordenação, formas de criptografia e modelos de aprendizado de máquina quânticos, áreas que abordaremos nos próximos capítulos.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br

REFERÊNCIAS

BASICS of Quantum Information. **IBM Quantum Platform**, 2025. Disponível em: https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information. Acesso em: 15 set. 2025.

DECOODT, P. W state in multi-qubit systems. [S.l.]: **GitHub**, 2018. Disponível em: https://github.com/qiskit-community/qiskit-community-tutorials/blob/master/awards/teach_me_qiskit_2018/w_state/W%20State%201%20-%20Multi-Qubit%20Systems.ipynb. Acesso em: 15 set. 2025.

FOUNDATIONS of quantum error correction. **IBM Quantum Platform**, 2025. Disponível em: https://learning.quantum.ibm.com/course/foundations-of-quantum-error-correction. Acesso em: 15 set. 2025.

GRIFFITHS, D. J. **Introduction to Quantum Mechanics**. 2nd ed. Cambridge: Cambridge University Press, 2017.

LOPES, A. VEDRAL, V. **Computação Quântica:** Da teoria à implementação. São Paulo: Blucher, 2019.

NIELSEN, M. A.; CHUANG, I. L. **Quantum Computation and Quantum Information**. 10. ed. Cambridge: Cambridge University Press, 2010.

QISKIT SDK API Documentation. **IBM Quantum Platform**, 2025. Disponível em: https://quantum.cloud.ibm.com/docs/en/api/qiskit. Acesso em: 25 set. 2025.

SMOLIN, J. A.; DIVINCENZO, D. P. Five two-bit quantum gates are sufficient to implement the quantum Fredkin gate. **Physical Review A**, v. 53, n. 4, p. 2855-2856, abr. 1996. Disponível em: https://link.aps.org/doi/10.1103/PhysRevA.53.2855. Acesso em: 15 set. 2025.

WILDE, M. M. **Quantum Information Theory**. 2. ed. Cambridge: Cambridge University Press, 2017.

YELEKAR, P. R.; CHIWANDE, S. S. **Introduction to reversible logic gates & its application**. *In*: 2nd National Conference on Information and Communication Technology - NCICT, 2 ed., 2011, Nagpur. Proceedings. Nagpur: International Journal of Computer Applications (IJCA), 2011. p. 5-9.

FIAP

PDF exclusivo para Leonardo Ruiz Orabona - pf1820
profleonardo.orabona@fiap.com.br